

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

HTTP Requests

Comunicación remota

En la actualidad, la mayoría de los programas que usamos se comunican con otros programas que corren en otros dispositivos.



Ejemplos: chat, páginas web, streaming de video, audio, e-commerce, geolocalización, redes sociales, juegos, videoconferencias, etc.

Comunicación remota

Ejemplo: correo postal



Pero..., ¿cómo llega la carta?

Comunicación remota

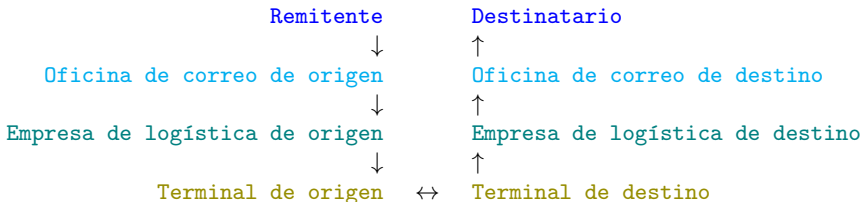
Ejemplo: correo postal



¡Hay todo un detrás de escena!

Comunicación remota

Ejemplo: correo postal



- En el origen, cada *nivel* delega la responsabilidad del envío en el *nivel inferior*.
- En el destino, cada *nivel* le avisa al *nivel superior* que “le llegó algo que tiene que entregar”.

En este modelo de capas, un nivel sólo se comunica en forma directa con su nivel inferior y su nivel superior.

Comunicación remota

Ejemplo: correo postal

Remitente ↔ Destinatario

Oficina de correo de origen ↔ Oficina de correo de destino

Empresa de logística de origen ↔ Empresa de logística de destino

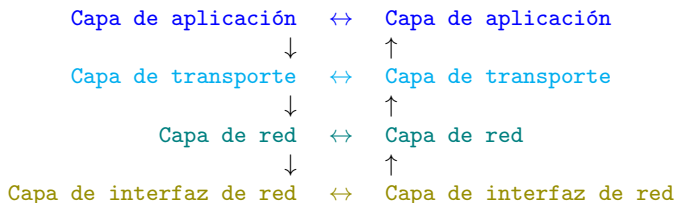
Terminal de origen ↔ Terminal de destino

Cada nivel maneja los mismos **protocolos** (*hablan el mismo idioma*) que su contraparte remota:

- ▶ El remitente y el destino saben escribir y leer cartas, meterlas en sobres, pegar sellos, etc.
- ▶ Las oficinas de correo saben ordenar muchas cartas de acuerdo al destino, suministrar empaquetados, etc.
- ▶ Las empresas de logística...

Modelo TCP/IP

El modelo **TCP/IP** (*Transmission Control Protocol/Internet Protocol*) es un modelo de capas que define un conjunto de **protocolos** para transmitir datos entre distintas computadoras a través de una red.



Modelo TCP/IP

El modelo **TCP/IP** (*Transmission Control Protocol/Internet Protocol*) es un modelo de capas que define un conjunto de **protocolos** para transmitir datos entre distintas computadoras a través de una red.

Capa de aplicación ↔ Capa de aplicación

Capa de transporte ↔ Capa de transporte

Capa de red ↔ Capa de red

Capa de interfaz de red ↔ Capa de interfaz de red

Protocolos:

- ▶ Aplicación: HTTP, SMTP, Telnet, FTP, etc.
- ▶ Transporte: TCP, UDP, etc.
- ▶ Red: IP, ARP, IGMP, ICMP, etc.
- ▶ Interfaz de red: ETHERNET, L2TP, etc.

Protocolo HTTP

HTTP (*Hypertext Transfer Protocol*) es un protocolo de comunicación que permite las transferencias de información a través de archivos (HTML, XLM, etc.) en Internet.



Protocolo HTTP



¿Qué es lo que escribimos en la barra de direcciones de un *browser*?

Una **URL** (*Uniform Resource Locator*) es una referencia a un recurso que especifica su ubicación en una red informática y un protocolo para recuperarlo:

`protocolo://host/carpeta1/carpeta2/.../carpetan/recurso`

Ejemplo:

`https://www.ole.com.ar/tema/lionel-messi.html`

- ▶ Protocolo: `https`
- ▶ Host: `www.ole.com.ar`
- ▶ Ubicación en el host: `tema/`
- ▶ Recurso: `archivo lionel-messi.html`

Nota: El protocolo `https` se diferencia de `http` en que encripta los datos antes de enviarlos por la red.

Biblioteca requests

Para comunicarnos con hosts remotos y pedirles recursos vamos a utilizar la biblioteca requests de Python.

```
import requests

def obtener_texto():
    url = "https://www...."
    response = requests.get(url)
    return response.text

input("Presiona Enter para descargar el texto")
texto = obtener_texto()
print("\nTexto recibido:")
print(texto)
```

- ▶ La función `requests.get(url)` envía una solicitud HTTP para obtener el recurso indicado por `url` y devuelve la respuesta del servidor.
- ▶ El resultado es un objeto, instancia de la clase `requests.Response`.
- ▶ Los objetos `Response` tienen un atributo `text` que contiene el texto devuelto por el servidor.

Biblioteca requests

Ejercicio 1.

Lorem ipsum es un texto de relleno usado habitualmente en diseño y tipografía para simular el contenido real de un documento o página web. Su origen se remonta a un texto en latín clásico, modificado para que no tenga significado, pero conserve una distribución de letras similar a la del lenguaje natural, lo que permite apreciar la forma y el aspecto del texto sin distraerse con el contenido:

```
 Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod  
tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,  
quis nostrud...
```

Los primeros fragmentos de este texto están disponibles en el archivo:

<https://www.learningcontainer.com/wp-content/uploads/2020/04/sample-text-file.txt>

Se pide escribir un programa en Python que: *(i)* utilice la librería `requests` para solicitar el recurso referenciado; y *(ii)* muestre por consola el texto recibido.

Biblioteca requests

En muchos casos, la respuesta del servidor remoto no está pensada para ser leída directamente por una persona. Por ejemplo, los archivos con extensión `.html` (o `.htm`) son textos que los navegadores web (como Chrome o Firefox) interpretan para mostrar una página en pantalla.

```
import requests

x = requests.get('https://w3schools.com/python/demopage.htm')
print(x.text)
```

```
<!DOCTYPE html>
<html>
  <body>
    <h1>This is a Test Page</h1>
  </body>
</html>
```

Biblioteca requests

En muchos casos no se solicita un archivo específico, sino que se invoca una funcionalidad provista por una [API](#) (*Application Programming Interface*) que corre en el servidor remoto.

Una API es, esencialmente, una biblioteca de funciones disponibles para su uso, y la respuesta que devuelve depende de cómo haya sido programada, sin seguir una estructura fija.

Ejemplo:

```
import requests

def obtener_chiste():
    url = "https://official-joke-api.appspot.com/random_joke"
    response = requests.get(url)
    return response.text

# Cuerpo principal del programa
input("Presioná Enter para obtener un chiste.")
chiste = obtener_chiste()
print("\nChiste recibido:")
print(chiste)
```

```
Chiste recibido:
{"type": "general",
 "setup": "Want to hear a joke about construction?",
 "punchline": "Nah, I'm still working on it.",
 "id": 151}
```

Biblioteca requests

Es muy habitual que el resultado de una función de una [API](#) tenga formato [JSON](#). La clase `Response` tiene un método `res.json()` que, en caso de que la respuesta de la API tenga este formato, transforma su contenido en objetos Python del mismo modo que lo hacía la biblioteca [json](#).

Ayudamemoria:

JSON	Python
String	<code>str</code>
Número (entero)	<code>int</code>
Número (real)	<code>float</code>
Array	<code>list</code>
Booleano (true/false)	<code>bool (True/False)</code>
Objeto JSON	<code>dict</code>
<code>null</code>	<code>None</code>

Biblioteca requests

Es muy habitual que el resultado de una función de una [API](#) tenga formato [JSON](#). La clase `Response` tiene un método `res.json()` que, en caso de que la respuesta de la API tenga este formato, transforma su contenido en objetos Python del mismo modo que lo hacía la biblioteca [json](#).

Ejemplo:

```
import requests

def obtener_chiste():
    url = "https://official-joke-api.appspot.com/random_joke"
    response = requests.get(url)
    data = response.json()

    chiste = data["setup"] + " " + data["punchline"]
    return chiste

# Cuerpo principal del programa
input("Presioná Enter para obtener un chiste.")
chiste = obtener_chiste()
print("\nChiste recibido:")
print(chiste)
```

```
Chiste recibido:
Why don't you find hippopotamuses hiding in trees? They're really good at it.
```


Biblioteca requests

Ejercicio 2.

Advice Slip es una API pública que devuelve consejos aleatorios sobre distintos temas de la vida cotidiana. La consulta se realiza mediante un requerimiento HTTP. La respuesta de la API es un objeto JSON que incluye, entre otros campos, la clave `advice` cuyo valor asociado contiene el consejo solicitado.

El *endpoint* básico es <https://api.advice Slip.com/advice>. La respuesta de la API es un objeto JSON que tiene la siguiente estructura aproximada:

```
{
  "slip": {
    "id": 42,
    "advice": "Don't worry if you make mistakes."
  }
}
```

Se pide escribir un programa en Python que: (i) utilice la librería `requests` para consultar la API pública de *Advice Slip*; y (ii) muestre por consola el consejo recibido en el JSON bajo la clave `advice`.

Repaso de la clase de hoy

- ▶ Redes
- ▶ Modelo de capas
- ▶ HTTP
- ▶ Biblioteca requests

Con los contenidos vistos hasta ahora, ya pueden completar hasta el ejercicio 2.2 (inclusive) de la guía de Métodos Analíticos.